

Merge Sort Complete DSA Notes (Divide and Conquer)

1. What is Merge Sort?

Merge Sort is a Divide and Conquer sorting algorithm.

Main idea:

1. Divide array into smaller halves.
2. Recursively sort both halves.
3. Merge sorted halves.

Example:

[5,2,8,1]

Divide:

[5,2] [8,1]

[5][2][8][1]

Merge:

[2,5]

[1,8]

Final:

[1,2,5,8]

2. Divide and Conquer Pattern

Merge Sort follows:

Divide:

Break problem into smaller problems.

Conquer:

Solve recursively.

Combine:
Merge solutions.

This pattern appears in:

- Binary Search
- Quick Sort
- Segment Trees
- Recursion problems

3. Merge Operation

Core part of merge sort.

Given:

left = [1,5,8]

right = [2,3,9]

Use two pointers.

Compare elements and put smaller first.

Result:

[1,2,3,5,8,9]

4. Recursion Tree

Every level divides array by 2.

Height:

$\log n$

At every level:

n elements are merged.

Total:

$n * \log n$

5. Time Complexity

Best Case:

$O(n \log n)$

Average Case:

$O(n \log n)$

Worst Case:

$O(n \log n)$

Space:

$O(n)$

6. Stability

Merge Sort is stable.

Meaning:

Equal elements maintain original order.

Useful when sorting objects.

Python Merge Sort

```
def mergeSort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    mid = len(arr) // 2  
  
    left = mergeSort(arr[:mid])  
    right = mergeSort(arr[mid:])
```

```

        return merge(left,right)

def merge(left,right):
    result=[]
    i=j=0

    while i<len(left) and j<len(right):
        if left[i]<=right[j]:
            result.append(left[i])
            i+=1
        else:
            result.append(right[j])
            j+=1

    result.extend(left[i:])
    result.extend(right[j:])

    return result

```

C++ Merge Sort

```

void merge(vector<int>& a,int l,int m,int r){
    vector<int> temp;

    int i=l;
    int j=m+1;

    while(i<=m && j<=r){
        if(a[i]<=a[j])
            temp.push_back(a[i++]);
        else
            temp.push_back(a[j++]);
    }

    while(i<=m)
        temp.push_back(a[i++]);

    while(j<=r)
        temp.push_back(a[j++]);

    for(int k=0;k<temp.size();k++)
        a[l+k]=temp[k];
    }

void mergeSort(vector<int>& a,int l,int r){
    if(l>=r)

```

```

return;

int mid=(l+r)/2;

mergeSort(a, l, mid);
mergeSort(a, mid+1, r);

merge(a, l, mid, r);
}

```

Java Merge Sort

```

class MergeSort{

void mergeSort(int[] arr,int l,int r){
    if(l>=r)
        return;

    int mid=(l+r)/2;

    mergeSort(arr, l, mid);
    mergeSort(arr, mid+1, r);

    merge(arr, l, mid, r);
}

void merge(int[] arr,int l,int m,int r){
    int[] temp=new int[r-l+1];

    int i=l;
    int j=m+1;
    int k=0;

    while(i<=m && j<=r){
        if(arr[i]<=arr[j])
            temp[k++]=arr[i++];
        else
            temp[k++]=arr[j++];
    }

    while(i<=m)
        temp[k++]=arr[i++];

    while(j<=r)
        temp[k++]=arr[j++];

    for(int x=0;x<temp.length;x++)
        arr[l+x]=temp[x];
}
}

```

```
}  
}
```

7. Merge Sort vs Quick Sort

Merge Sort:

Time:
Always $O(n \log n)$

Space:
 $O(n)$

Stable:
Yes

Quick Sort:

Average:
 $O(n \log n)$

Worst:
 $O(n^2)$

Space:
 $O(\log n)$

Stable:
No

8. Advanced Applications

Merge Sort concepts are used in:

1. Count Inversions
2. Reverse Pairs
3. External Sorting
4. Sorting Linked Lists
5. Divide and Conquer DP
6. Segment Trees

9. Important Leetcode Problems

LC 912:
Sort an Array

LC 148:
Sort List

LC 315:
Count Smaller Numbers After Self

LC 493:
Reverse Pairs

LC 327:
Count Range Sum

Pattern Recognition

Think Merge Sort when:

- Need stable sorting
- Need divide and conquer
- Need count pairs while sorting
- Need process left and right halves efficiently